

Controlling the Unitree Quadrupe

From SportClient basics to camera feeds to a vision-language-action model (NaVILA)

`run_sports_client_test.py` · `run_camera_test.py` · *NaVILA live client*

Recap: NaVILA turns everyday directions into robot movement

A beginner's view of a vision-language-action system for legged robots

- NaVILA is an AI navigation system that helps a legged robot understand where to go using a human instruction and the live video from its camera.

1 SEE

The robot receives a human instruction (text) plus 8 camera images sampled from its run, always including the latest frame.



2 UNDERSTAND

A vision-language-action model turns both into simple navigation commands.



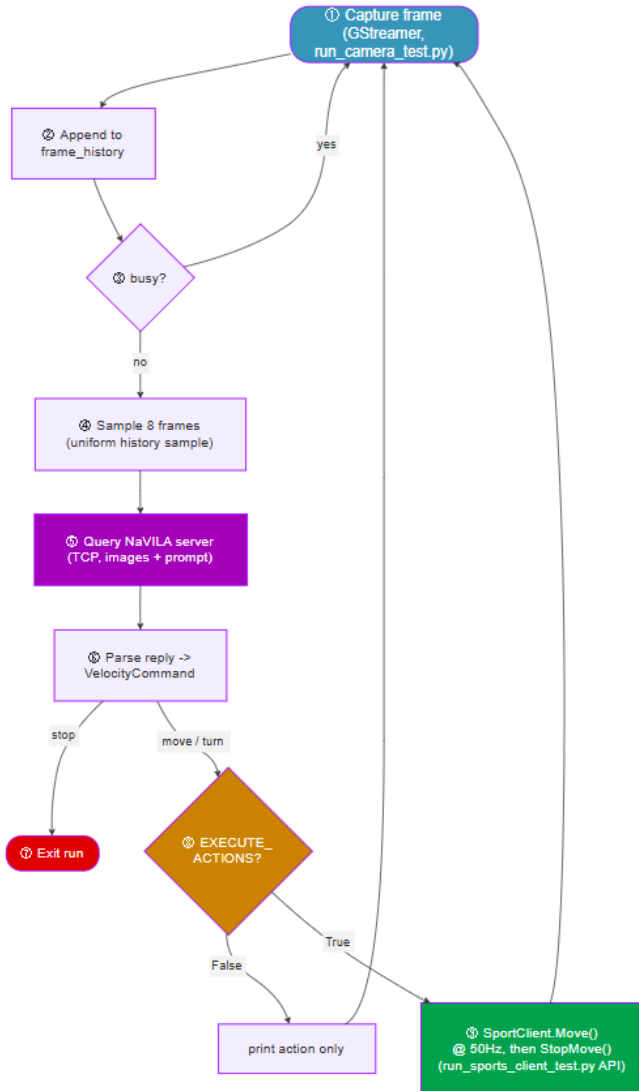
3 MOVE

A real-time locomotion policy carries out the commands while avoiding obstacles.

In simple terms: you describe the route in natural language; NaVILA links those words and what the robot sees to safe legged movement.

Recap: NaVILA — Detailed Control Loop

navila_live_client.py: two machines — robot (capture + motors) <-> NaVILA server (inference)



- ① Capture frame — `cap.read()` via the same GStreamer pipeline as `run_camera_test.py`
- ② Append to frame_history — `frame_to_base64_jpg(frame)`; list grows for the whole run (full history, never trimmed)
- ③ busy? — `runner.is_busy()`: skip this tick if the previous inference/action is still running
- ④ Sample 8 frames — `sample_from_history()`: black-pad if < 8 , indices = $\text{int}(i * (\text{count} - 1) / 7)$ for i in $0..6$, + latest frame
- ⑤ Query NaVILA server — `query_navila()`: TCP to NAVILA_HOST:NAVILA_PORT with the 8 JPEGs + PROMPT
- ⑥ Parse reply — `parse_velocity_command()`: regex for stop / move forward Xcm / turn left|right Xdeg -> VelocityCommand
- ⑦ Exit run — `cmd.stop` sets `_should_exit`; main loop condition ends the script
- ⑧ EXECUTE_ACTIONS? — safety gate: False (default) = print only, True = drive motors, no human per action
- ⑨ `SportClient.Move()` — `execute_command()`: 50Hz `Move(vx,vy,wz)` loop for duration_s (same API as `run_sports_client_test.py`), finally: `StopMove()`

Run: `robot-env/.venv/bin/python navila_live_client.py` (server must already be running)



Controlling Motion with SportClient

Part 1: what it is, and why you must SSH into the robot

SportClient is Unitree's high-level motion API (unitree_sdk2py)

One call = one command: BalanceStand(), StopMove(), Move(vx, vy, vyaw), GetState(), FrontFlip(), etc.

Why SSH is required:

The robot's motion controller only talks over its internal "br0" network

Your laptop cannot reach it directly — code must run ON the robot's onboard computer

Workflow: ssh into the robot → activate robot-env venv → run the script there

```
ssh unitree@192.168.50.181
cd ~/Desktop/a2_explorations
source robot-env/.venv/bin/activate
python run_sports_client_test.py
```

ChannelFactoryInitialize(0, "br0")

Must be called once before creating SportClient() — it sets up the DDS communication channel over the br0 interface.

run_sports_client_test.py — Interactive Command Menu

Part 2: try commands live from the terminal

Script prints a menu; type a name or id, e.g. "balance_stand" or "1"

Common commands to try in order:

balance_stand (id 1) — stand up and become movable

move (id 5) — Move(vx, vy, vyaw); must be re-sent at 50Hz to keep moving

get_state (id 8) — check fsm_name: STAND_DOWN / STAND_UP / NORMAL_WALK

stop_move (id 2) — stop safely

Try: list → balance_stand → get_state → move → stop_move

run_camera_test.py — Reading the Onboard Camera

GStreamer + OpenCV, no SDK needed

The A2 streams H.264 video over UDP multicast on its br0 network

OpenCV can read it directly via a GStreamer pipeline string — no Unitree SDK needed for this part

Must use the robot-env venv's cv2 (built with GStreamer support), not a plain "pip install opencv-python"

```
gstreamer_str = (  
    "udpsrc address=230.1.1.1 port=1720 multicast-iface=br0 "  
    "! application/x-rtp, media=video, encoding-name=H264 "  
    "! rtph264depay ! h264parse ! avdec_h264 ! videoconvert "  
    "! video/x-raw,width=1280,height=720,format=BGR ! appsink drop=1"  
)  
cap = cv2.VideoCapture(gstreamer_str, cv2.CAP_GSTREAMER)  
while cap.isOpened():  
    ret, frame = cap.read()  
    if ret:  
        cv2.imshow("Input via Gstreamer", frame)
```




Move past the brown chair, turn towards the plants and stop

Putting It All Together

run_sports_client_test.py — the motion "alphabet": one SportClient call per action, run over SSH on the robot

run_camera_test.py — the robot's "eyes": GStreamer pipeline gives you raw camera frames as a plain cv2 feed

navila_live_client.py — combines both: camera frames feed a VLM, the VLM's answer drives the SportClient

Next steps for students:

- For those selected for finals, do not be afraid of significant added complexity when moving code to robot